BlockBoost: Scalable and Efficient Blocking through Boosting

Anonymous Author
Anonymous Institution

Abstract

As datasets grow larger, matching and merging entries from different databases has become a costly task in modern data pipelines. To avoid expensive comparisons between entries, blocking similar items is a popular preprocessing step. In this paper, we introduce BlockBoost, a novel boosting-based method that generates compact binary hash codes for database entries, through which blocking can be performed efficiently. The algorithm is fast and scalable, resulting in computational costs that are orders of magnitude lower than current benchmarks. Unlike existing alternatives, BlockBoost comes with associated feature importance measures for interpretability, and possesses strong theoretical guarantees, including lower bounds on critical performance metrics like recall and reduction ratio. Finally, we show that BlockBoost delivers great empirical results, outperforming state-of-the-art blocking benchmarks in terms of both performance metrics and computational cost.

1 Introduction

With larger datasets and disparate data sources becoming more prevalent, properly identifying, integrating and linking entries across multiple datasets is now a crucial step in many data pipelines. This process of identifying and disambiguating entities within one or more datasets is known as entity matching, entity resolution or record linkage [1]. Applications include merging health records [2, 3], aggregating census data [4], identifying war casualties [5], detecting crimes [6], cataloging bibliographic citations or business products, and matching genome sequences [7].

Preliminary work. Under review by AISTATS 2024. Do not distribute.

A fundamental issue in entity matching is the quadratic number of comparisons necessary between database items. Blocking [8] is a popular technique to reduce the number of comparisons. It consists of blocking together items considered similar (in some metric) and only comparing entries within the same block. For example, if the goal is to match a list of customer purchase records to a list of customer accounts, blocks may be based on the customer's last name, or the ZIP code of their billing address, or all of these combined. If the blocks are well-crafted, this can significantly improve the speed and efficiency of the matching process.

A common way to create blocks is via hashing. A hash function yields a low-dimension binary representation of each entry, called the hash code. Unlike standard dimensionality-reduction methods, the fact that this representation is binary is important to ensure fast retrieval time, which is of the utmost importance for large databases [9, 10, 11]. An efficient hash code maps similar candidates to the same hash code and dissimilar items to different hash codes, significantly reducing the associated number of comparisons and ensuring that similar items are indeed between these candidates.

However, devising good hash functions can be very hard. An important technique to create hash functions is locality-sensitive hashing (LSH). In locality-sensitive hashing [9, 12], hash codes are built in such a way that points that are close in some metric typically have similar hash codes. It has many interesting theoretical guarantees that are valid under broad circumstances. However, the fact that LSH is agnostic to the nature of the underlying data often leads to suboptimal performance. To address this issue, there is a class of techniques known as learning to hash [13, 14, 11, 15] that aims to improve the hashing efficiency by learning hash functions tailored for specific tasks, such as entity matching [8]. While this approach can improve performance, it also introduces new challenges. For instance, methods like kernel LSH and TLSH maintain LSH's theoretical guarantees, but sacrifice scalability and efficiency [16, 17, 18]. A more recent alternative for blocking dispenses hashing altogether and is based

on deep learning embeddings applied to entity matching [19, 20]. These state-of-the-art solutions use neural networks to learn feature representations that capture the underlying relationships between records, thus improving the accuracy and effectiveness of blocking.

In this paper, we propose a new blocking method called *BlockBoost*, that combines a boosting step that learns a pairwise similarity function and a hashing step on top of which blocking can be quickly performed (see Figure 1). Since boosting is a fast machine learning method with great out-of-the-box performance, BlockBoost works well for many different types of unstructured data. Furthemore, unlike many traditional blocking alternatives, it is possible to devise lower bounds on BlockBoost's performance in terms of relevant metrics such as recall and reduction ratio. This results in a data-driven technique that is efficient and scalable, with provable guarantees. Finally, BlockBoost achieves superior results to state-of-the-art deep learning solutions on multiple datasets.

Main contributions. We introduce BlockBoost, a novel blocking algorithm that combines hashing and boosting with several features:

- Efficient data compression: by learning hash codes from the data, BlockBoost is able to extract and the combine the most distinguishing features and automatically pick the right hash size; e.g., in one of the empirical examples considered, it obtains state-of-the-art results by compressing 9600 bits in the original features into a 150-bit hash. Also, hashing dimensions are ordered by importance, and can be trimmed for further compression;
- Speed: the training is quasi-linear in the entries, with linear prediction time; it is an order of magnitude faster than alternatives, it scales to millions of entries and runs well on CPUs;
- Simple tuning: BlockBoost has a single, easy-to-interpret hyperparameter;
- Theoretical results: unlike most blocking algorithms, BlockBoost has theoretical guarantees on its performance and lower bounds on popular metrics such as recall and reduction ratio;
- Interpretability: it is possible to interpret the contribution of each data feature to the final hashes by looking at importance measures derived from boosting; this is useful to identify the most distinguishing features available in the data;
- Empirical performance: BlockBoost outperforms state-of-the-art solutions on many canonical blocking datasets in terms of recall, reduction ratio and their harmonic average.

2 Related work

Due to its importance in data pipelines across many applications, entity matching [21, 22, 23] is a widely studied field. While blocking is an old idea [24], it remains an active area of research [25].

Several blocking techniques are based on exact matches or certain blocking keys, such as attribute matching [26] and token blocking [27]. However, many real-world data possesses unnormalized or corrupted data, posing a serious challenge to such methods [28]. Because BlockBoost learns attributes through boosting, it does not suffer from this problem.

There are also methods that do not require exact attribute matching for block creation. For example, the canopy clustering algorithm [29] groups together items based on the similarity of certain fields using a clustering algorithm. However, it is slow and requires the tuning of several hyperparameters to obtain a competitive performance. BlockBoost, on the other hand, has a single hyperparameter.

Hashing-based blocking methods, such as LSH, are closer in spirit to our approach. They partition records to the same blocks if they share the same hash value, using a pre-specified random hash function. For example, in [8, 5], community detection techniques [18] and clustering algorithm [30] were used as a post-processing steps to LSH and lead to good results in entity matching. There are also learning to hash algorithms [13, 14, 11, 15] that try to learn the hash functions via a training stage. However, they are typically not employed for blocking due to scalability issues with learning overly complex algorithms [16, 17, 18]. BlockBoost, in contrast, can be orders of magnitude faster.

In recent years, there has been a growing interest in using deep learning embeddings to enhance the performance of blocking in entity matching [19, 20]. These approaches are oftentimes considered to be state-of-the-art, as they leverage neural networks to learn intricate feature representations that capture the inherent relationships between records.

Finally, we note the work of [31], which also uses boosting on top of hashing; however, its underlying idea is quite different from ours, as their method is tailored to the nearest neighbor problem [9, 32], and, unlike our method, relies on the existence of a distance function in a metric spaces.

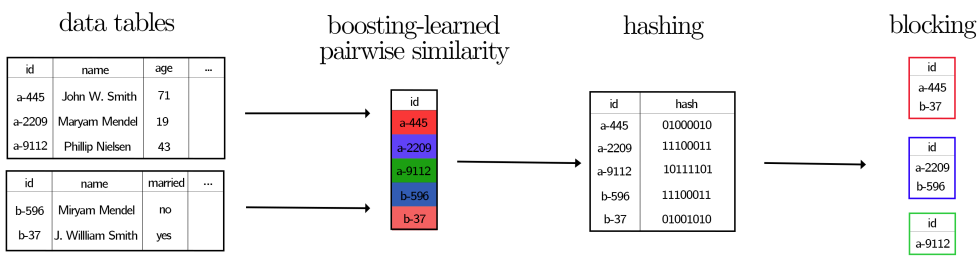


Figure 1: Overview of how BlockBoost performs blocking through (i) boosting and (ii) hashing.

3 BlockBoost

Given datasets $\mathcal{A} := \{A_\ell\}_{\ell=1}^{N_\mathcal{A}}$ and $\mathcal{B} := \{B_r\}_{r=1}^{N_\mathcal{B}}$, both contained in a set $\mathcal{X}$, and a relationship between items $\sim_R$, we want to find pairs (A_ℓ, B_r) such that $A_\ell \sim_R B_r$, where we assume $\sim_R$ is unknown and must be learned. In our entity matching application, A_ℓ and B_r will correspond to entities in different datasets and we will say that $A_\ell \sim_R B_r$ if and only if A_ℓ and B_r are the same entity. For ease of presentation, our setup assumes two databases; however, the results hold for an arbitrary number of data collections (or one collection with many representations of the same item).

Our goal is to build a hash table for these items so that, given an item A_ℓ, one can find $B_r \sim_R A_\ell$ with as few table lookups as possible. To this end, we suppose we have access to a training sample,

$$\mathcal{S}_{\text{train},n} := \{((A_i, B_i), y_i) \in (\mathcal{A} \times \mathcal{B}) \times \{-1, 1\}, i \in [n]\},$$

such that, $y_i = 1$ if $A_i \sim_R B_i$ and -1 otherwise. This sample will be used in a training stage so our algorithm can learn a similarity classifier via a sample of similar/dissimilar items using boosting.

Our method consists of two steps:

Boosting step. Using boosting, we learn binary classifiers $\{k_t^*\}_{t=1}^T$ over $\mathcal{X}$, as well as convex weights $\{\alpha_t^*\}_{t=1}^T$ for these classifiers. Then, given items $A \in \mathcal{A}$ and $B \in \mathcal{B}$, we construct a similarity function $f^*(A, B) = \sum_{t=1}^T \alpha_t^* k_t^*(A) k_t^*(B)$. In this step, weights α_t^* are expected to be large when the product $k_t^*(A)k_t^*(B)$ correlates strongly with the similarity relation $A \sim_R B$, and consequently $f^*(A, B)$ is close to $+1$ when $A \sim_R B$ and $f^*(A, B)$ is close to -1 when $A \not\sim_R B$.

Hashing step. From the $\{\alpha_t^*, k_t^*\}_{t=1}^T$ learned in the previous step, various techniques can be employed to create hash codes for blocking. In this paper, we focus on using the similarity function learned in the boosting to create blocks via a weighted hamming distance between pairs. The Supplementary Material discusses another plausible option based on LSH [9, 12].

We now consider each of the above steps in further detail.

3.1 Boosting step

Fix a family $\mathcal{K}$ of binary classifiers $k : \mathcal{X} \to \{-1, +1\}$ and a max number of iterations $T_{\max} \in \{1, 2, \dots\}$. To find the functions $\{k_t^*\}_{t=1}^T \in \mathcal{K}$ and the convex weights $\{\alpha_t^*\}_{t=1}^T$, with $T \leq T_{\max}$, we use a boosting algorithm over our training sample $\mathcal{S}_{\text{train},n} = ((A_i, B_i), y_i)_{i=1}^n$; see Algorithm 1. Note that T, rather than $T_{\max}$, is the key determinant of the total number of binary classifiers produced by our method. As we will demonstrate in forthcoming discussions, it also plays a pivotal role in determining the ultimate level of compression achieved by our method, as it directly influences the number of bits required for the hash.

While Algorithm 1 is reminiscent of AdaBoost [33], note we are optimizing a function that is quadratic over the chosen classifier. This is crucial for the hashing step, described in Section 3.2. Also, there are classifier families for which this optimization problem is feasible. An example is the set of decision stumps $\mathcal{H}_{\text{stumps}}$: for $x \in \mathbb{R}^d$, if $x_{(j)}$ indicates the j-th coordinate of $x \in \mathbb{R}^d$, then

$$\mathcal{H}_{\text{stumps}} = \left\{ \mathbf{1}_{[x_{(j)} < \xi]} \cup \mathbf{1}_{[x_{(j)} \geq \xi]} \ : \ \xi \in \mathbb{R}, j \in [p] \right\}. \tag{1}$$

Intuitively, since the model is trying to predict matches and non-matches, we expect that, given items $A \in \mathcal{A}$ and $B \in \mathcal{B}$, the learned function

$$f^*(A, B) = \sum_{t=1}^T \alpha_t^* k_t^*(A) k_t^*(B), \tag{2}$$

will be a good similarity measure between A and B. That is, $f^*(A, B)$ should be close to $+1$ when $A \sim_R B$, and close to -1 otherwise. Notice that, as usual in boosting, larger weights α_t^* are given to the classifiers k_t^* that achieve the smallest values of boosted errors ε_t. That is, our algorithm naturally gives more weight

Algorithm 1 Boosting step

Input: $\mathcal{S}_{\text{train},n} = ((A_i, B_i), y_i)_{i=1}^n$, max number of iterations $T_{\max} \in \mathbb{N}$, binary family $\mathcal{K}$

for $i \leftarrow 1$ to n **do**
  $Q_1(i) \leftarrow \frac{1}{n}$
end for
$t \leftarrow 1$
$T \leftarrow T_{\max}$
while $t \leq T_{\max}$ **do**
  $k_t^* \leftarrow$ classifier in $\mathcal{K}$ with smallest error $\varepsilon_t = \sum_{i=1}^n Q_t(i)\mathbf{1}_{[y_i k_t^*(A_i)k_t^*(B_i)<0]}$
  if $\varepsilon_t \geq 1/2$ **then**
    $T \leftarrow t-1$
    break
  else
    $\alpha_t' \leftarrow \frac{1}{2}\log\left(\frac{1-\varepsilon_t}{\varepsilon_t}\right)$
    $Z_t \leftarrow 2\left[\varepsilon_t(1-\varepsilon_t)\right]^{1/2}$
    for $i \leftarrow 1$ to n **do**
      $Q_{t+1}(i) \leftarrow \frac{Q_t(i)\exp\left(-\alpha_t' y_i k_t^*(A_i)k_t^*(B_i)\right)}{Z_t}$
    end for
  end if
  $t \leftarrow t+1$
end while
for $t \leftarrow 1$ to T **do**
  $\alpha_t^* \leftarrow \frac{\alpha_t'}{\sum_{s=1}^T \alpha_s'}$
end for

Output: $(\alpha_t^*)_{t=1}^T$, $(k_t^*)_{t=1}^T$

to functions $k_t^*(A)k_t^*(B)$ that correlate more strongly to the similarity relation.

3.2 Hashing step

We now use the convex weights $(\alpha_t^*)_{t=1}^T$ and the functions $(k_t^*)_{t=1}^T$ to construct hash functions that will be used for blocking. Our solution involves a straightforward calculation of a weighted Hamming distance over hash codes to create the blocks.

For each element $A \in \mathcal{X}$, we create a T-bit hash function g that is given by

$$g(A) = (k_1^*(A), \ldots, k_T^*(A)),$$

where T is the number of iterations used in the boosting step. Items (A, B) will be part of the same block if for a given small $\delta \in [0, 1]$,

$$f^*(A, B) = \sum_{i=1}^T \alpha_i^* k_t^*(A)k_t^*(B) \geq 1 - \delta \tag{3}$$

After creating the blocks, we can reduce direct comparisons only to pairs (A, B) in each block, sparing a number of unnecessary comparisons when declaring a match.

An additional benefit of the proposed hashing approach is that it allows for the expression in Equation 3 to be written as a weighted Hamming distance between the hashes $g^*(A)$ and $g^*(B)$,

$$\begin{aligned}\sum_{i=1}^T \alpha_i^* k_t^*(A)k_t^*(B) &= \sum_{i=1}^T \alpha_i^*(1 - |k_t^*(A) - k_t^*(B)|) \\ &= 1 - \sum_{i=1}^T \alpha_i^* |k_t^*(A) - k_t^*(B)|,\end{aligned}$$

since $\sum_{t=1}^T \alpha_t^* = 1$. Therefore, items (A, B) will be part of the same block only if

$$\sum_{i=1}^T \alpha_i^* |k_t^*(A) - k_t^*(B)| \leq \delta, \tag{4}$$

and this can be computed efficiently, as we discuss in Section 5.6.

4 Theoretical guarantees

For our theoretical analysis, we assume we have two datasets $\mathcal{A} := \{A_\ell\}_{\ell=1}^{N_\mathcal{A}}$ and $\mathcal{B} := \{B_r\}_{r=1}^{N_\mathcal{B}}$, both contained in a set $\mathcal{X}$ and a notion of similarity $\sim_R$ between items (A_ℓ, B_r). For entity matching problems, we assume that $A_\ell \sim_R B_r$ if and only if A_ℓ and B_r are the same entity. All proofs for the results in this section can be found in the Supplementary Material.

4.1 Performance metrics

We first define traditional performance metrics for entity matching. The goal of our method is to ensure that, for each $A \in \mathcal{A}$, one can find all similar $B \in \mathcal{B}$ while doing as few pairwise comparisons as possible. This is made precise by the Recall and the Reduction Ratio (RR) metrics [1]:

$$\text{Recall} := \frac{1}{|\mathcal{M}|}\sum_{(\ell,r)\in\mathcal{M}} \mathbf{1}_{[A_\ell \text{ and } B_r \text{ share a block}]}; \tag{5}$$

$$\text{RR} := 1 - \frac{1}{|\mathcal{N}|}\sum_{(\ell,r)\in\mathcal{N}} \mathbf{1}_{[A_\ell \text{ and } B_r \text{ share a block}]}, \tag{6}$$

where $\mathcal{N} := [N_\mathcal{A}] \times [N_\mathcal{B}]$ denotes all possible pairs and $\mathcal{M}$ denotes the set of matching pairs:

$$\mathcal{M} := \{(\ell, r) \in \mathcal{N}, A_\ell \sim_R B_r, (A_\ell, B_r) \in \mathcal{A} \times \mathcal{B}\}. \tag{7}$$

Recall, also known as pair completeness, measures the proportion of similar pairs that end up in the same block, whereas RR, also known as efficiency, measures the proportion of the $N_\mathcal{A} \cdot N_\mathcal{B}$ potential pairwise comparisons that are avoided. Ideally, we would like to find as many as possible matching pairs (Recall ≈ 1), while avoiding as many comparisons as possible (RR ≈ 1).

To be able to compare performance through a single real value, a commonly used metric in the blocking literature [26] is the the harmonic mean between Recall and RR, $\mathrm{H}(\mathrm{Recall}, \mathrm{RR})$, given by:

$$\mathrm{H}(\mathrm{Recall}, \mathrm{RR}) := 2 \cdot \frac{\mathrm{Recall} \cdot \mathrm{RR}}{\mathrm{Recall} + \mathrm{RR}}. \tag{8}$$

4.2 Performance guarantees

The notion of margin plays a central role in our analysis. It corresponds to our intuition that f^* is a good estimator of the similarity relation $\sim_R$ if it confidently identifies matches and non-matches.

Definition 4.1 (θ-margin condition)**.** For a fixed $\theta > 0$, given classifiers $\{k_t^*\}_{t=1}^T$ and convex weights $\{\alpha_t^*\}_{t=1}^T$ we say that the similarity function f^* defined in (2) has θ-margin if, with probability at least $1 - \eta$ over the choice of (A, B, y), it holds that

$$\begin{aligned} f^*(A, B) &> +\theta, \ \text{ if } \ y = +1, \quad \text{and} \\ f^*(A, B) &< -\theta, \ \text{ if } \ y = -1. \end{aligned}$$

Our first theorem shows that the function f^* obtained in the boosting step via Algorithm 1, satisfies the θ-margin condition with high probability. The value of η depends on the Rademacher complexity [34] of the base classifiers $\mathcal{K}$ and the samples $\mathcal{S}_{\mathcal{A},n} := \{A_i\}_{i=1}^n$ and $\mathcal{S}_{\mathcal{B},n} := \{B_i\}_{i=1}^n$ given by

$$\begin{aligned} \mathfrak{R}_{\mathcal{S}_{\mathcal{A},n}}(\mathcal{K}) &= \frac{1}{n}\mathbb{E}_\sigma \left[\sup_{k \in \mathcal{K}} \sum_{i=1}^n \sigma_i y_i k(A_i) \right], \\ \mathfrak{R}_{\mathcal{S}_{\mathcal{B},n}}(\mathcal{K}) &= \frac{1}{n}\mathbb{E}_\sigma \left[\sup_{k \in \mathcal{K}} \sum_{i=1}^n \sigma_i y_i k(B_i) \right], \end{aligned}$$

where $\sigma_1, \ldots, \sigma_n$ are independent Rademacher random variables, i.e., uniformly chosen in $\{-1, 1\}$.

Theorem 4.2 (Performance of the boosting step)**.** *With probability at least $1 - \delta$, the function f^* corresponding to the output of Algorithm 1 satisfies the θ-margin condition with the value of $\eta := \eta_{train}(f^*, \mathcal{S}_{train,n}, \theta, \delta)$ given by:*

$$\begin{aligned} \eta := 2^T \prod_{t=1}^T \varepsilon_t^{1/2-\theta}(1 - \varepsilon_t)^{\theta - 1/2} \\ + \frac{8}{\theta}\left(\mathfrak{R}_{\mathcal{S}_{\mathcal{A},n}}(\mathcal{K}) + \mathfrak{R}_{\mathcal{S}_{\mathcal{B},n}}(\mathcal{K})\right) + \sqrt{\frac{\log(1/\delta)}{2n}}, \end{aligned}$$

where $\varepsilon_t > 0$ are the errors defined in Algorithm 1.

Furthermore, if there exists $\gamma > 0$ such that for all $t \in [T]$, $\gamma \leq (1/2 - \varepsilon_t)$ and $\theta \leq 2\gamma$, then the first term in the right-hand side above decreases exponentially with T.

Intuitively, the product term in the definition of $\eta_{\mathrm{train}}(f, \mathcal{S}_{\mathrm{train},n}, \theta, \delta)$ is a margin bound over the training data. When $\varepsilon_t \leq 1/2 - \gamma$ for all t, the term will decay exponentially fast in T for suitable choices of margin parameters θ. The other terms in $\eta_{\mathrm{train}}(f, \mathcal{S}_{\mathrm{train},n}, \theta, \delta)$ correspond to a generalization bound used for the test error.

Our next result shows that when Definition 4.1 holds, then the proposed hashing steps of BlockBoost produce high values of the Recall and RR metrics in expectation, with a suitable choice of hyperparameters.

Theorem 4.3 (Performance Weighted Hamming distance hashing)**.** *Consider databases $\mathcal{A}$ and $\mathcal{B}$ such that $|\mathcal{A}| = N_\mathcal{A}$, $|\mathcal{B}| = N_\mathcal{B}$ and let $\mathcal{M}$ be the set of matching pairs as in 7. For given $\theta > 0$ if the output f^* of Algorithm 1 satisfies the θ-margin condition and we set $1 - \theta > \delta$ in 3.2, then BlockBoost achieves*

$$\begin{aligned} \mathbb{E}\left[\mathrm{RR}\right] &\geq (1 - \eta)\left(1 - \frac{|\mathcal{M}|}{N_\mathcal{A} \cdot N_\mathcal{B}}\right), \\ \mathbb{E}\left[\mathrm{Recall}\right] &\geq 1 - \eta \end{aligned}$$

where expectations are with respect to the training step, and Recall and RR are defined in (5) and (6).

Note that, in entity matching problems, it is usually the case that $|\mathcal{M}| \ll N_\mathcal{A} \cdot N_\mathcal{B}$. Thus, the expected RR is close to 1, meaning that only a few comparisons have to be made.

4.3 Algorithmic complexity and speed

We now analyze the algoritmic complexity and speed of BlockBoost in each step.

Boosting. We use stump functions (1) as the family of base classifiers. As described in [35], to determine the stump with the minimal weighted error at each round of boosting we can presort each component in $\mathcal{O}(n \log n)$ time with a total computational cost of $\mathcal{O}(nd \log n)$. For a given component, there are only $n+1$ possible distinct thresholds, since two thresholds between the same consecutive component values are equivalent. To find the best threshold at each round of boosting, all of these possible $n + 1$ values can be compared, which can be done in $\mathcal{O}(n)$ time. Thus, the total computational complexity of the algorithm for T rounds of boosting is $\mathcal{O}(nd \log n + ndT)$.

Hashing and blocking. Our binary hashing, with their low number of bits, allow for faster construction of candidate pairs compared to floating-point vectorizations. As a comparison, DeepBlocker's algorithm [19], one of the fastest blocking alternatives, uses a NVIDIA V100 GPU and the well-optimized FAISS library, and still took 34 minutes for a dataset of 1

million entries [36]. In contrast, BlockBoost achieved the same empirical performance in 2 minutes using a consumer-grade i7 CPU.

5 Experiments

The code to replicate the experiments in this section are available in the Supplementary Material, as well as in the repository located at `https://github.com/redacted_repository`.

5.1 Datasets

We make use of canonical blocking datasets that are sourced from a broad spectrum of domains and span a wide range of sizes as shown in Table 1. They are all publicly available and have been used in previous work on entity matching [8, 5, 19, 1, 7, 37]. Further datasets are considered in the Supplementary Material.

The datasets are divided into train, validation, and test sets, with 15%, 15%, and 70% of the total entries, respectively. Additionally, we ensure that the distribution of matches in each fold aligns with these proportions. The train set is used for models that require a training phase and the validation set is employed for adjusting hyperparameters, as needed. Note BlockBoost only has the single parameter δ in (4); in practice, T is chosen large enough that Algorithm 1 hits the stopping condition in line 8. Lastly, the test set is utilized to evaluate the performance metrics defined in Section 4.1.

We opted to use small training and validation proportions to more accurately represent a real-world scenario where a blocking model must be applied, but there are limited labeled examples available.

5.2 Benchmark models

BlockBoost is compared against the following well-known methods for blocking in entity matching.

- *Canopy* [29]: groups records into blocks based on the similarity of certain fields, using a clustering algorithm. For our experiments, we utilized the implementation in [38].
- *K-Means locality-sensitive hashing (KLSH)* [30]: uses k-means algorithm to construct a low-dimensional projection of the data. We use the code in [39] for the algorithm.
- *Transitive locality-sensitive hashing (TLSH)* [18] uses a community detection technique to find similar entities. Our code follows [40], which implements the work [8].
- *Spectral hashing (Spect)* [41]: this learn to hash method uses a graph partitioning relaxation that is closely related to semantic hashing. We use the implementation in [42].
- *AGHasher (AG)* [43]: a learn to hash method automatically finds compact hash codes using graph-based neighborhood structure in the data. Our experiments follow the code in [44].
- *DeepBlocker* [19]: a state-of-the-art blocking algorithm for entity matching using deep learning. We included the three distinct neural network architectures available as individual models: CTT, Autoencoder (AE), and Hybrid, which integrates both the CTT and Autoencoder models. We use the official code repository provided by the authors [36].

5.3 Vectorization

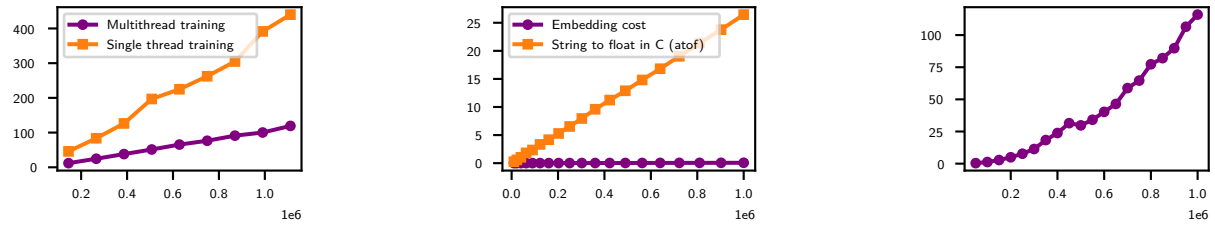
To ensure consistency with the original implementations of the baseline blocking models, we employed two distinct types of vectorization.

Shingling and MinHash. This vectorization is used for Canopy, TLSH and KSLH baseline models, as described in [8, 5] and implemented in [39, 40]. We first apply the shingling technique to construct a sparse numerical representation of our textual data. Then, we apply the MinHash algorithm [45, 8] to transform our sparse numerical information into a dense one.

SIF Embedding. We utilized this vectorization technique for both DeepBlocker and our own model, BlockBoost. SIF (Smooth Inverse Frequency) embedding [46] is a vector space model commonly used in NLP tasks, which assigns weights to each word vector based on their frequency in a reference corpus. This weighting scheme allows the resulting document vectors to capture the semantic meaning of the text while reducing the impact of frequent but less informative words.

5.4 Training setup

For most of our benchmark models, training is performed on the training data fold. The exception is DeepBlocker, which does not require training data to work and instead relies on generating artificial training data. In particular, entities in the test set are compared against other, distinct entities in the test set (these will be assumed to be non-matches) or against a slightly perturbed version of the original entity (these will be declared matches). This allows the method to train on a significantly larger number of examples. Further details regarding this methodology are explained in detail in [19]. BlockBoost works with either of these



(a) Training cost in second. (b) Embedding cost in seconds. (c) Blocking cost in seconds.

Figure 2: BlockBooster's cost in seconds on an i7 processor, as the size of the artificially created dataset based on `restaurant` gets larger. For $n = 10^6$, BlockBoost's blocking step takes at most 116 seconds, while DeepBlocker's CTT takes over 34 minutes on an NVIDIA V100 GPU.

Table 1: Baseline datasets for blocking, and compression (in bits) achieved by BlockBoost's hashes over the original set of features

DATASET	ENTITIES	MATCHES	TABLES	FEATURES	BLOCKBOOST BIT COMPRESSION
ABT_BUY	2,173	1,097	2	4	9×
AMZ_GG	4,589	1,300	2	5	23×
DBLP_ACM	4,908	2,224	2	4	8×
DBLP_SCH	66,879	5,347	2	4	5×
RESTAURANT	865	752	1	4	2×
RLDATA500	500	50	1	8	7×
RLDATA10K	10,000	1,000	1	8	6×
MUSICBRAINZ	19,375	10,000	5	7	3×
WM_AMZ	24,583	1,145	2	29	101×

training setups. Below, we mimic DeepBlocker's approach so our method also does not require training data to work. We employ a ratio of 16 negative examples to each positive example in all our experiments.

5.5 Blocking performance

Table 2 displays the performance of BlockBoost against all the benchmarks on the datasets from Table 1. Note each of DeepBlocker's possible configurations (CTT, AE, Hybrid) are considered separately. Overall, BlockBoost (BB) displays the best average harmonic mean of Recall and RR, and even when it is not the top-performing algorithm, it is generally close to it. The Supplementary Material includes the individual values of Recall and RR out of which the harmonic mean was derived. Note that only DeepBlocker, in its best possible network configuration, was able to rival BlockBoost's performance.

5.6 Computational performance

Even when BlockBoost is trained on millions of pairs, it only takes a couple of minutes on a modern CPU, and, as Figure 2a shows, it scales well as n gets larger. This is crucial for a blocking algorithm, since the number of pairs to be matched in real-world examples can often become unwieldy (and are, after all, the main motivation for blocking algorithms). After that, the process of mapping entries from the initial vectorization to the binary embedding is so fast that it's dominated by the low level C function `atof`, which converts text to floats from the csv file (see Figure 2b).

The binary nature of our embedding, coupled with the low number of bits that the boosting produces, means that the set of candidate pairs can be constructed much faster than it would be possible with floating-point based vectorizations (see Table 1 for the bit compression Blockboost's hashes achieve versus the original feature set on each dataset). As a consequence, BlockBoost on CPU achieves a 17× speedup over DeepBlocker using an NVIDIA V100 GPU. To make sense of this, note that our binary embeddings, evaluated at Table 2, have an average size of 158 bits, and a maximum size of 256 bits. This is particularly useful for any x86 CPU with SSE4, since they can compute the number of 1s in a word in a single cycle. Modern GPUs also have instructions dedicated to this operation.

Finally, since the dimensions of the embedding are ordered by importance, the least significant bits can be trimmed to make the operations even faster.

Table 2: Comparison table of the evaluation metric H(Recall, RR), defined in 8, with the best model per dataset in bold. BlockBoost (BB) has the best performance overall.

Dataset	BB	Canopy	KLSH	TLSH	Spect	AG	CTT	AE	Hybrid
ABT_BUY	**0.911**	0.761	0.365	0.625	0.263	0.503	0.907	0.817	0.822
AMZ_GG	**0.877**	0.605	0.515	0.281	0.518	0.539	0.810	0.849	0.849
DBLP_ACM	0.993	0.850	0.895	0.861	0.662	0.696	0.993	0.996	**0.998**
DBLP_SCH	0.989	0.891	0.691	0.543	0.602	0.670	**0.991**	0.980	0.983
RESTAURANT	0.988	0.785	0.937	0.838	0.519	0.728	**0.997**	0.997	0.997
RLDATA500	**0.992**	0.829	0.969	0.982	0.691	0.717	0.966	0.966	0.966
RLDATA10K	**0.999**	0.929	0.926	0.987	0.755	0.800	0.957	0.928	0.926
MUSICBRAINZ	0.991	0.101	0.944	0.950	0.662	0.737	**0.994**	0.992	0.992
WM_AMZ	**0.943**	0.017	0.495	0.005	0.577	0.558	**0.943**	0.917	0.942
AVERAGE	**0.965**	0.641	0.749	0.675	0.583	0.660	0.951	0.938	0.942

5.7 Interpretability

At each iteration $t = 1, \ldots, T$ of the boosting process, BlockBoost produces a stump function k_t^* and a weight α_t^*. The weight can be interpreted as an indication of how significant the stump is for matching entities. Additionally, as the stump is defined by a projected feature j_t^* and a threshold ξ_t^*, the weight also indicates how strongly such feature correlates with the similarity relation between items. Thus, the α_t^* can be understood as giving an automatic feature importance.

This can be very helpful when interpreting which features carry most of the signal for finding matches. This also allows for discarding irrelevant features for added speed, as well as simple attribute matching based on the selected ones. As an example, Figure 3 shows the importance of some features in the `musicbrainz` dataset when each one is vectorized separately. Here, importance is understood by the sum of weights associated to each feature.

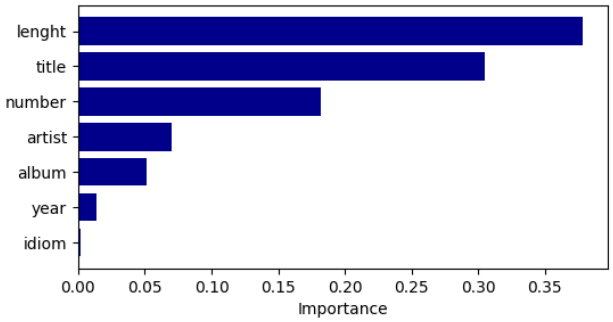


Figure 3: Feature relevance identified by BlockBoost during the boosting step for the `musicbrainz` dataset.

Intuitively, if someone intends to correlate songs from various databases, BlockBoost considers the length and title as the most critical attributes. The track number on a CD emerges as a surprisingly reliable third option. This is because it's a numeric value and is typically automatically extracted, whereas other entries involve textual data and potential manual input. Notably, by generating hashes exclusively from the concatenation of the length and number (both numeric fields, although not an immediately obvious choice), one can achieve a high blocking performance with a H(Recall, RR) close to 0.91.

6 Conclusion

This paper introduces BlockBoost, a novel blocking method for entity resolution that is data-driven, efficient, and scalable. The results show that combining boosting techniques with hashing leads to a powerful blocking method that empirically outperforms state-of-the-art hashing and deep learning models in terms of scalability and performance over several canonical blocking datasets.

Beyond speed and efficiency, the algorithm is also interpretable and theoretically sound. It outputs weights that reflect variable importance measures, providing users with insight into the most relevant data features for the blocking process. BlockBoost also comes with guaranteed lower bounds on each performance through an extension of margin theory results.

Finally, BlockBoost is able to generate hashes whose size are automatically determined in a data-dependent fashion, leading to great compression and fast retrieval time. Since the dimensions of the hash are ordered by importance, one can trim the hashes for added time and better compression.

As a possible future avenue for this line of research, we believe BlockBoost can be adapted to many other applications beyond entity matching, such as image retrieval and similarity search, while carrying the same advantages as presented above.

References

[1] Peter Christen. A survey of indexing techniques for scalable record linkage and deduplication. *IEEE Transactions on Knowledge and Data Engineering*, 24(9):1537–1555, 2012.

[2] D E Clark. Practical introduction to record linkage for injury research. *Injury Prevention*, 10(3):186–191, 2004.

[3] Chris Kelman, John Bass, and C.D.J. Holman. Research use of linked health data - a best practice protocol. *Australian and New Zealand journal of public health*, 26:251–5, 02 2002.

[4] William E Winkler. Overview of record linkage and current research directions. Technical report, BUREAU OF THE CENSUS, 2006.

[5] Rebecca C. Steorts and Anshumali Shrivastava. Probabilistic blocking with an application to the syrian conflict. 2018.

[6] Jeff Jonas and Jim C. Harper. Effective counterterrorism and the limited role of predictive data mining. 2006.

[7] Peter Christen. *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Springer Publishing Company, Incorporated, 2012.

[8] Rebecca C. Steorts, Samuel L. Ventura, Mauricio Sadinle, and Stephen E. Fienberg. A comparison of blocking methods for record linkage, 2014.

[9] Alexandr Andoni and Piotr Indyk. Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions. In *2006 47th Annual IEEE Symposium on Foundations of Computer Science (FOCS'06)*, pages 459–468, 2006.

[10] Moses S. Charikar. Similarity estimation techniques from rounding algorithms. In *Proceedings of the Thiry-Fourth Annual ACM Symposium on Theory of Computing*, STOC '02, page 380388, New York, NY, USA, 2002. Association for Computing Machinery.

[11] Brian Kulis and Trevor Darrell. Learning to hash with binary reconstructive embeddings. In Y. Bengio, D. Schuurmans, J. Lafferty, C. Williams, and A. Culotta, editors, *Advances in Neural Information Processing Systems*, volume 22. Curran Associates, Inc., 2009.

[12] Sariel Har-Peled, Piotr Indyk, and Rajeev Motwani. Approximate nearest neighbor: Towards removing the curse of dimensionality. *Theory of Computing*, 8(14):321–350, 2012.

[13] Alexandr Andoni and Daniel Beaglehole. Learning to hash robustly, guaranteed, 2021.

[14] Yair Weiss, Antonio Torralba, and Rob Fergus. Spectral hashing. In D. Koller, D. Schuurmans, Y. Bengio, and L. Bottou, editors, *Advances in Neural Information Processing Systems*, volume 21. Curran Associates, Inc., 2008.

[15] Jingdong Wang, Ting Zhang, jingkuan song, Nicu Sebe, and Heng Tao Shen. A survey on learning to hash. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(4):769–790, 2018.

[16] Brian Kulis and Kristen Grauman. Kernelized locality-sensitive hashing for scalable image search. In *2009 IEEE 12th International Conference on Computer Vision*, pages 2130–2137, 2009.

[17] Ke Jiang, Qichao Que, and Brian Kulis. Revisiting kernelized locality-sensitive hashing for improved large-scale image retrieval. 11 2014.

[18] Jonathan Oliver, Chun Cheng, and Yanggui Chen. Tlsh – a locality sensitive hash. In *2013 Fourth Cybercrime and Trustworthy Computing Workshop*, pages 7–13, 2013.

[19] Saravanan Thirumuruganathan, Han Li, Nan Tang, Mourad Ouzzani, Yash Govind, Derek Paulsen, Glenn Fung, and AnHai Doan. Deep learning for blocking in entity matching: A design space exploration. *Proc. VLDB Endow.*, 14(11):24592472, 10 2021.

[20] Sidharth Mudgal, Han Li, Theodoros Rekatsinas, AnHai Doan, Youngchoon Park, Ganesh Krishnan, Rohit Deep, Esteban Arcaute, and Vijay Raghavendra. Deep learning for entity matching: A design space exploration. In *Proceedings of the 2018 International Conference on Management of Data*, SIGMOD '18, page 1934, New York, NY, USA, 2018. Association for Computing Machinery.

[21] William E. Winkler. Methods for evaluating and creating data quality. *Information Systems*, 29(7):531–550, 2004. Data Quality in Cooperative Information Systems.

[22] Vassilis Christophides, Vasilis Efthymiou, Themis Palpanas, George Papadakis, and Kostas Stefanidis. An overview of end-to-end entity resolution for big data. *ACM Comput. Surv.*, 53(6), 12 2020.

[23] Ahmed K. Elmagarmid, Panagiotis G. Ipeirotis, and Vassilios S. Verykios. Duplicate record detection: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 19(1):1–16, 2007.

[24] Ivan P. Fellegi and Alan B. Sunter. A theory for record linkage. *Journal of the American Statistical Association*, 64(328):1183–1210, 1969.

[25] George Papadakis, Dimitrios Skoutas, Emmanouil Thanos, and Themis Palpanas. Blocking and filtering techniques for entity resolution: A survey. *ACM Comput. Surv.*, 53(2), 3 2020.

[26] Fabio Azzalini, Songle Jin, Marco Renzi, and Letizia Tanca. Blocking techniques for entity linkage: A semantics-based approach. *Data Sci. Eng.*, 6:20–38, 2020.

[27] Kevin O'Hare, Anna Jurek-Loughrey, and Cassio de Campos. *A Review of Unsupervised and Semi-supervised Blocking Methods for Record Linkage*, pages 79–105. Springer International Publishing, Cham, 2019.

[28] Wei Zhang, Hao Wei, Bunyamin Sisman, Xin Luna Dong, Christos Faloutsos, and Davd Page. AutoBlock. In *Proceedings of the 13th International Conference on Web Search and Data Mining*. ACM, 1 2020.

[29] Andrew McCallum, Kamal Nigam, and Lyle H. Ungar. Efficient clustering of high-dimensional data sets with application to reference matching. In *Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '00, page 169178, New York, NY, USA, 2000. Association for Computing Machinery.

[30] Loïc Paulevé, Hervé Jégou, and Laurent Amsaleg. Locality sensitive hashing: a comparison of hash function types and querying mechanisms. *Pattern Recognition Letters*, 31(11):1348–1358, August 2010.

[31] Sunwoo Kim, Haici Yang, and Minje Kim. Boosted locality sensitive hashing: Discriminative binary codes for source separation. In *ICASSP 2020 - 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 106–110, 2020.

[32] Nitin Bhatia and Vandana. Survey of nearest neighbor techniques. 2010.

[33] Yoav Freund and Robert E Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *J. Comput. Syst. Sci.*, 55(1):119–139, August 1997.

[34] Peter Bartlett and Shahar Mendelson. Rademacher and Gaussian complexities: Risk bounds and structural results. *Journal of Machine Learning Research*, 3(Nov):463–482, 2002.

[35] Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar. *Foundations of Machine Learning*. The MIT Press, 2012.

[36] Saravanan Thirumuruganathan, Han Li, Nan Tang, Mourad Ouzzani, Yash Govind, Derek Paulsen, Glenn Fung, and AnHai Doan. Repository: Deepblocker. `https://github.com/qcri/DeepBlocker`, 2021.

[37] Hanna Köpcke, Andreas Thor, and Erhard Rahm. Evaluation of entity resolution approaches on real-world match problems. *Proc. VLDB Endow.*, 3(12):484493, 9 2010.

[38] Repository: Canopy. `https://github.com/AlanConstantine/CanopyByPython`, 2018.

[39] Repository: Klsh. `https://github.com/cleanzr/klsh`, 2020.

[40] Repository: Tlsh. `https://github.com/cleanzr/tlsh`, 2020.

[41] Yair Weiss, Antonio Torralba, and Rob Fergus. Spectral hashing. *Advances in neural information processing systems*, 21, 2008.

[42] Repository: Learnhash. `https://github.com/galad-loth/LearnHash`, 2018.

[43] Wei Liu, Jun Wang, Sanjiv Kumar, Shih-Fu Chang, and T Scheffer. Hashing with graphs proceedings of the 28th international conference on machine learning. *ICML 2011*, pages 1–8, 2011.

[44] Repository: Aghasher. `https://github.com/dstein64/aghasher`, 2022.

[45] Andrei Z Broder, Moses Charikar, Alan M Frieze, and Michael Mitzenmacher. Min-wise independent permutations. *Journal of Computer and System Sciences*, 60(3):630–659, 2000.

[46] Sanjeev Arora, Yingyu Liang, and Tengyu Ma. A simple but tough-to-beat baseline for sentence embeddings. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017.

Checklist

1. For all models and algorithms presented, check if you include:
   (a) A clear description of the mathematical setting, assumptions, algorithm, and/or model. [✓**Yes**/No/Not Applicable]
   (b) An analysis of the properties and complexity (time, space, sample size) of any algorithm. [✓**Yes**/No/Not Applicable]
   (c) (Optional) Anonymized source code, with specification of all dependencies, including external libraries. [✓**Yes**/No/Not Applicable]

2. For any theoretical claim, check if you include:
   (a) Statements of the full set of assumptions of all theoretical results. [✓**Yes**/No/Not Applicable]
   (b) Complete proofs of all theoretical results. [✓**Yes**/No/Not Applicable]
   (c) Clear explanations of any assumptions. [✓**Yes**/No/Not Applicable]

3. For all figures and tables that present empirical results, check if you include:
   (a) The code, data, and instructions needed to reproduce the main experimental results (either in the supplemental material or as a URL). [✓**Yes**/No/Not Applicable]
   (b) All the training details (e.g., data splits, hyperparameters, how they were chosen). [✓**Yes**/No/Not Applicable]
   (c) A clear definition of the specific measure or statistics and error bars (e.g., with respect to the random seed after running experiments multiple times). [✓**Yes**/No/Not Applicable]
   (d) A description of the computing infrastructure used. (e.g., type of GPUs, internal cluster, or cloud provider). [✓**Yes**/No/Not Applicable]

4. If you are using existing assets (e.g., code, data, models) or curating/releasing new assets, check if you include:
   (a) Citations of the creator If your work uses existing assets. [✓**Yes**/No/Not Applicable]
   (b) The license information of the assets, if applicable. [✓**Yes**/No/Not Applicable]
   (c) New assets either in the supplemental material or as a URL, if applicable. [✓**Yes**/No/Not Applicable]
   (d) Information about consent from data providers/curators. [Yes/No/✓**Not Applicable**]
   (e) Discussion of sensible content if applicable, e.g., personally identifiable information or offensive content. [Yes/No/✓**Not Applicable**]

5. If you used crowdsourcing or conducted research with human subjects, check if you include:
   (a) The full text of instructions given to participants and screenshots. [Yes/No/✓**Not Applicable**]
   (b) Descriptions of potential participant risks, with links to Institutional Review Board (IRB) approvals if applicable. [Yes/No/✓**Not Applicable**]
   (c) The estimated hourly wage paid to participants and the total amount spent on participant compensation. [Yes/No/✓**Not Applicable**]